GSSS INSTITUTE OF ENGINEERING AND TECHNOLOGY FOR WOMEN

(GSSSIETW), MYSURU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

MLOps ASSIGNMENT – 2

TRACKING A PRE-TRAINED SENTIMENT MODEL WITH MLFLOW

Submitted by	E. Sai Sruthi
USN	4GW23CI014
Department	CSE (AI & ML)
Semester	VII
Academic year	2026–2027

Submitted in partial fulfilment of the requirements of the MLOps Laboratory

Academic Year 2026–2027

1. AIM

To use a pre-trained sentiment analysis model and track its inference experiments using MLflow. The experiments are performed with different batch sizes (1, 8 and 4), and their parameters, metrics and prediction results are recorded and compared.

2. SOFTWARE REQUIREMENTS

- Operating System: Windows
- Programming Language: Python 3.10
- IDE: Visual Studio Code
- Libraries: MLflow, Transformers, PyTorch, Pandas and NumPy
- Pre-trained model: distilbert-base-uncased-finetuned-sst-2-english

3. INTRODUCTION

MLOps (Machine Learning Operations) is a set of practices used to manage machine learning models and their workflows. It helps organise experiments, record results and make model development and deployment easier to manage.

MLflow is an open-source platform used to track machine learning experiments. It can record parameters (settings used in an experiment), metrics (measured results) and artifacts (files produced during an experiment). Its user interface allows different runs to be viewed and compared.

In this assignment, a pre-trained DistilBERT sentiment analysis model is used. The model classifies a sentence as POSITIVE or NEGATIVE. No model training is performed; only inference is carried out on ten labelled sentences.

4. PROCEDURE

1. Create a project folder and open it in Visual Studio Code.
2. Create and activate a Python 3.10 virtual environment.
3. Install MLflow, Transformers, PyTorch, Pandas and NumPy.
4. Load the pre-trained DistilBERT sentiment analysis model.
5. Prepare ten sentences with their actual sentiment labels.
6. Run inference using batch sizes 1, 8 and 4.
7. For each run, log the model name, batch size and number of samples as parameters.
8. Calculate and log accuracy, average confidence and total inference latency as metrics.
9. Save the input sentences, actual labels, predicted labels and confidence scores in predictions.csv and log it as an artifact.
10. Start the MLflow user interface, view the experiment runs and compare their results.

5. EXPERIMENT DETAILS

Experiment name: sentiment-inference-lab

Model: distilbert-base-uncased-finetuned-sst-2-english

Number of test sentences: 10

Batch sizes tested: 1, 8 and 4

Parameters logged: model_name, batch_size and num_samples

Metrics logged: accuracy, avg_confidence and total_latency_sec

Artifact logged: predictions.csv

6. PYTHON IMPLEMENTATION

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE")
]

texts = [t for t, _ in data]
labels = [l for _, l in data]

mlflow.set_experiment("sentiment-inference-lab")

def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)

    with mlflow.start_run(run_name=run_name):
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]
        accuracy = sum(
            p == l for p, l in zip(pred_labels, labels)
        ) / len(labels)
        avg_conf = sum(p["score"] for p in preds) / len(preds)

        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)

        df = pd.DataFrame({
```

```
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds]
        })
        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy,
              "latency:", round(latency, 3))

MODEL = "distilbert-base-uncased-finetuned-sst-2-english"

run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")
```

7. EXPERIMENTAL RESULTS

The following values are from the latest successful execution recorded during the experiment. The latency can vary slightly between executions because it depends on system load and runtime conditions.

Run	Batch size	Accuracy	Inference latency (sec)
run_batch1	1	1.00 (100%)	0.254
run_batch8	8	1.00 (100%)	0.065
run_batch4	4	1.00 (100%)	0.074

Note: The experiment was executed more than once, so MLflow contains duplicate runs for each batch size. The table above shows one set of latest recorded results for comparison.

8. MLFLOW USER INTERFACE AND ARTIFACTS

The MLflow UI was opened locally at http://127.0.0.1:5000. The experiment sentiment-inference-lab displayed the recorded runs. The comparison view was used to compare the parameters and metrics of the runs. The predictions.csv artifact was also opened and checked. It contains the columns text, actual, predicted and confidence.

Figure 1: MLflow experiment runs

[Insert the screenshot showing run_batch1, run_batch8 and run_batch4.]

Figure 2: Compare runs

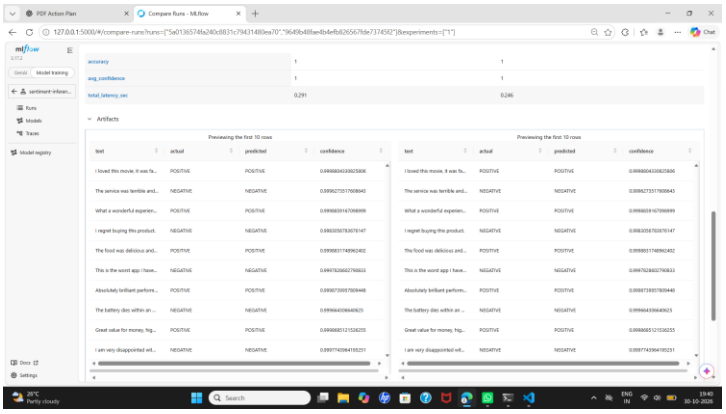
[Insert the screenshot showing batch_size and the logged metrics.]

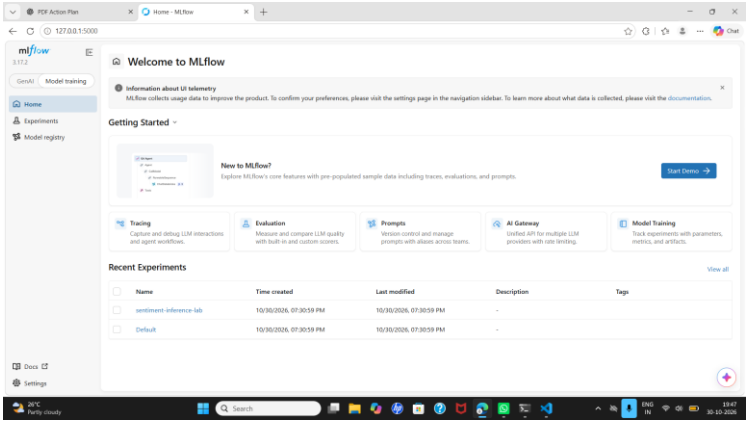
Figure 3: predictions.csv artifact

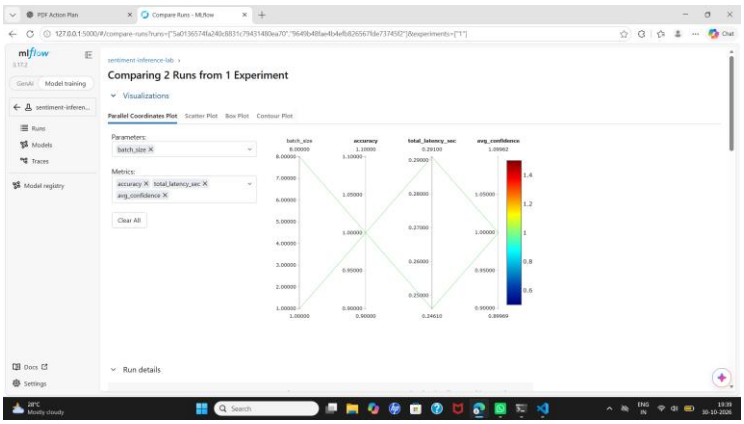
[Insert the screenshot showing the CSV columns and prediction rows.]

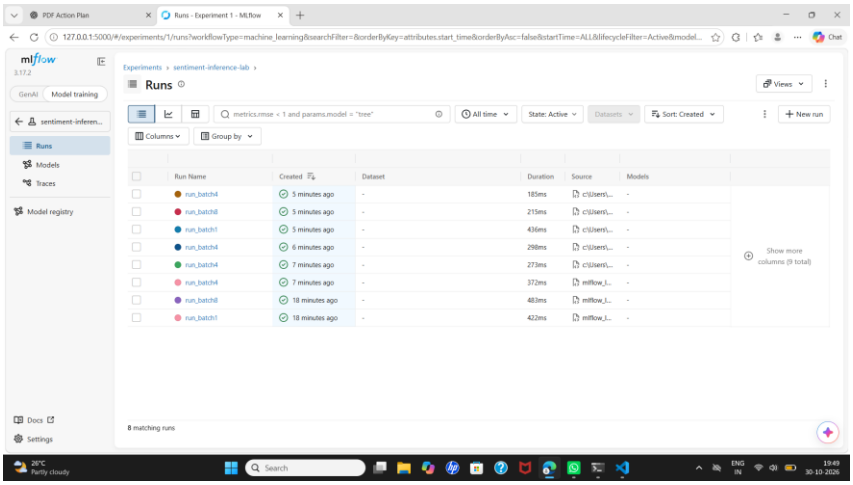
9. OBSERVATIONS

- All three batch sizes achieved 100% accuracy on the ten test sentences.
- Batch size 8 recorded the lowest inference latency (0.065 seconds) in the latest execution.
- The accuracy remained the same for batch sizes 1, 4 and 8 in this experiment.
- Inference latency may vary between executions due to system and runtime conditions.







- 

10. CONCLUSION

The pre-trained DistilBERT sentiment analysis model was successfully used to perform sentiment inference with batch sizes 1, 8 and 4. MLflow recorded the experiment parameters, metrics and predictions.csv artifact. The runs were viewed and compared using the MLflow UI. In the latest execution, batch size 8 gave the lowest inference latency while maintaining 100% accuracy.